

ANTIGRADIANT — материалы для сайта

WIUT Hackathon 2026 · Computer Vision track · всё из реальных данных нашей модели и сэмплов

Этот документ — для того, кто делает сайт. В папке `final/ANTIGRADIANT_site_kit/` лежат все файлы, о которых здесь идёт речь: видео с разметкой, JSON с событиями и кривыми риска, графики EDA, кадры примеров и ошибок, демо-приложение. Главное правило: на сайте только реальные данные. Жюри проверяет «failures stated plainly» и «results readable and correct»; выдуманные цифры и фиктивный live-demo опаснее, чем честные скромные результаты.

Раздел сайта (вес)	Что взять из кита	Статус
Live demo (30%)	<code>demo/</code> — настоящее приложение (Gradio), запускает нашу модель на загруженном видео; <code>DEPLOY.txt</code> — как выложить на Hugging Face Spaces и встроить в сайт	заменить мок
Sample videos (20%)	<code>videos/annotated_C38xx.mp4</code> (4 видео) + <code>data/events_*.json</code> + <code>data/risk_*.json</code>	заменить мок
EDA (15%)	<code>eda/*.png</code> и <code>*.json</code> , <code>eda/findings_ru.md</code> (выводы, повлиявшие на решение)	заменить проценты-заглушки
Approach + report (15%)	раздел 3 и 6 этого PDF: схема, learned vs rule-based, что сработало/нет	дописать
Team (10%)	как есть; проверить ссылки GitHub/LinkedIn	ок
Design, extras (10%)	интерактивные графики из JSON, клик по событию → перемотка, ablation-таблица с реальными числами (раздел 5)	дописать

1. Что на сайте сейчас неправда и что поставить вместо

Сейчас на сайте	Как на самом деле (ставить это)
Классы <code>following_too_close</code> , <code>speeding</code> , <code>solid_line</code>	Таких классов нет. Официальные 14: <code>accident</code> , <code>near_miss</code> , <code>red_light</code> , <code>wrong_way</code> , <code>illegal_u_turn</code> , <code>stopped_vehicle</code> , <code>jaywalking</code> , <code>failure_to_yield</code> , <code>illegal_turn</code> , <code>solid_line_crossing</code> , <code>stop_line</code> , <code>congestion</code> , <code>road_obstacle</code> , <code>fire_smoke</code> . Наша модель выдаёт 7: <code>red_light</code> , <code>stop_line</code> , <code>jaywalking</code> , <code>failure_to_yield</code> , <code>stopped_vehicle</code> , <code>congestion</code> , <code>solid_line_crossing</code>
«25 FPS»	Видео 3840×2160, 29.97 fps (в PDF ошибка), H.264 4:2:2 10-bit ~140 Мбит/с
«Разбор <code>camera.md</code> », формула гомографии к плоскости дороги	<code>camera.md</code> нам не дали. Наша гомография — регистрация кадра на опорный вид (SIFT + RANSAC), потому что камеру сдвигали на 50–100 px между записями. Геометрия сцены (зебры, островки, стоп-линия, сплошные) нарисована вручную в CVAT на опорном кадре
«CLANE-предобработка» в конвейере	CLANE используется только внутри регистрации (SIFT на выровненной яркости). Детектор получает кадр как есть
Проценты 14 классов (42.4% <code>congestion</code> и т.д.)	Реальная разметка команды: таблица в разделе 4 (число событий по классам в 4 видео)
Ablation: mAP 78.4%, 44.8 FPS, Anticipation AP 0.762, RT-DETR, Swin3D...	Этих экспериментов не было. Реальные абляции — раздел 5 (декодирование, разрешение, детектор, правила, Part B)
«Пробовали 3D CNN / optical flow / LSTM»	Не пробовали. Реальные неудачи — раздел 6
Live demo: события и кривая риска при загрузке берутся из массива/синусоиды	Так нельзя: жюри загрузит своё видео. Подключить <code>demo/</code> (настоящий инференс на CPU) или честно подписать
Таймлайн C3905 и кривая риска, набранные руками	<code>data/events_C3905.json</code> и <code>data/risk_C3905.json</code> — выход модели
Ссылка на репозиторий Hackathon-WIUT, релиз весов	Код модели: <code>github.com/AsanAshirov/WestCV</code> (rer v1.0.0). Веса скачивает <code>weights/download.sh</code> с официального релиза Ultralytics (<code>yolo26m.pt</code> , 44 МБ). Одна команда — один репозиторий: договоритесь, какой ссылкой сдаём

Если чего-то нет в этом документе — лучше не писать, чем придумать.

2. Команда (для README и сайта)

Участник	Роль	Что сделал
Азам Ходжиметов	Team lead, web platform	Сайт, live demo, координация
Диёра Фатахова	Data & annotation lead	Разметка сэмплов в CVAT, EDA
Асан Аширов	ML & computer vision	Конвейер, правила, светофоры, Part B, оценка, пакет сдачи

Проверьте роли и напишите имена латиницей так же, как на сайте: в README сейчас Azam Khodjimetov, Diyora Fatakhova, Asan Ashirov.

3. Подход: схема и что learned, а что rule-based

Схема для блока Pipeline (можно нарисовать SVG по этому тексту):

video.mp4 (3840×2160, 29.97 fps)
→ PyAV: декодируем только опорные кадры (каждый 3-й), уменьшаем до 1280 px
→ YOLO26m (COCO, без дообучения): люди, машины, автобусы, двухколёсные
→ ByteTrack: треки (люди и транспорт отдельно)
→ медианный фон → SIFT + RANSAC: привязка к опорному виду → зебры, островки, тротуары, стоп-линия, сплошные линии в координатах этого видео
→ светофоры: 2 головы, смотрящие в камеру → состояние ламп по цвету (порог на видео)
→ Part A: правила по 7 классам в единицах сцены (высоты рамок, секунды) → склейка сегментов → события
→ Part B (причинно, кадр за кадром): детектор на каждом 6-м кадре (640 px) → онлайн-трекинг → время до столкновения + резкое торможение → риск 0...1

Компонент	Learned / rule-based	Почему
Детекция участников движения	learned: YOLO26m, веса COCO как есть	Людей находим с recall ~97%; дообучение не нужно, узкое место не здесь
Трекинг	алгоритм: ByteTrack	Стандарт, быстрый, без обучения
Привязка сцены	алгоритм: SIFT + RANSAC	Камеру сдвигали на 50–100 px; без неё зоны «уезжают»
Светофоры	правила: цвет ламп + порог Otsu на видео	Цикл фиксированный (75/80 с), читается из пикселей
События (7 классов)	правила на треках и геометрии	Разметки мало (4 видео): правила прозрачны, настраиваются по нашей разметке
Part B риск	правила: TTC и торможение	В сэмплах нет аварий — обучать не на чем; PDF сам советует TTC

4. EDA сэмплов (реальные графики — папка eda/)

Dataset passport: 4 sample clips, one fixed Sony camera over a signalised intersection (2026-09-18)

C3896 · midday



C3897 · midday



C3902 · evening (sunset)



C3905 · evening (sunset)



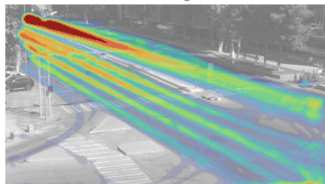
	C3896	C3897	C3902	C3905
Resolution	3840x2160	3840x2160	3840x2160	3840x2160
Frame rate (fps)	29.97	29.97	29.97	29.97
Frames	10200	9525	9525	3825
Duration (s)	340.3	317.8	317.8	127.6
Time of day	midday	midday	evening (sunset)	evening (sunset)
Brightness, 0-255	95.7	96.2	62.8	38.5
Red / blue ratio	1.043	1.048	0.635	0.668
Vehicle tracks (>=)	434	428	511	227
Person tracks (>=)	580	479	794	339
Detections / frame	55.7	56.9	67.0	62.3
Persons in vehicle	5.6	2.2	2.4	3.5

All clips: H.264 High 4:2:2 10-bit, ~140 Mbit/s, GOP IBBP, LPCM audio. Detector YOLO26m @ 1280 px on every 3rd frame, conf >= 0.3.

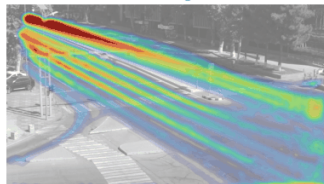
Паспорт датасета: 4 видео, 18.4 мин, 3840×2160, 29.97 fps, день и закат.

Motion heatmaps: moving vehicles (top) and pedestrians (bottom)

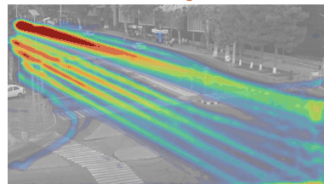
C3896 · moving vehicles



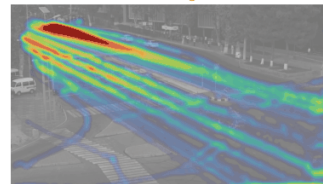
C3897 · moving vehicles



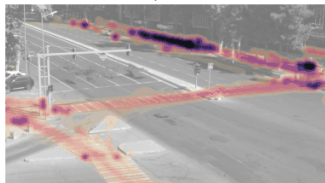
C3902 · moving vehicles



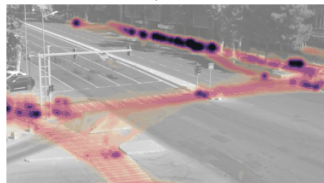
C3905 · moving vehicles



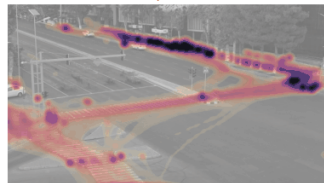
C3896 · pedestrians



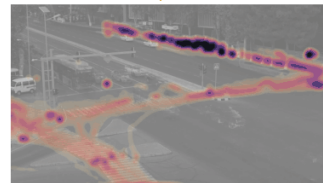
C3897 · pedestrians



C3902 · pedestrians



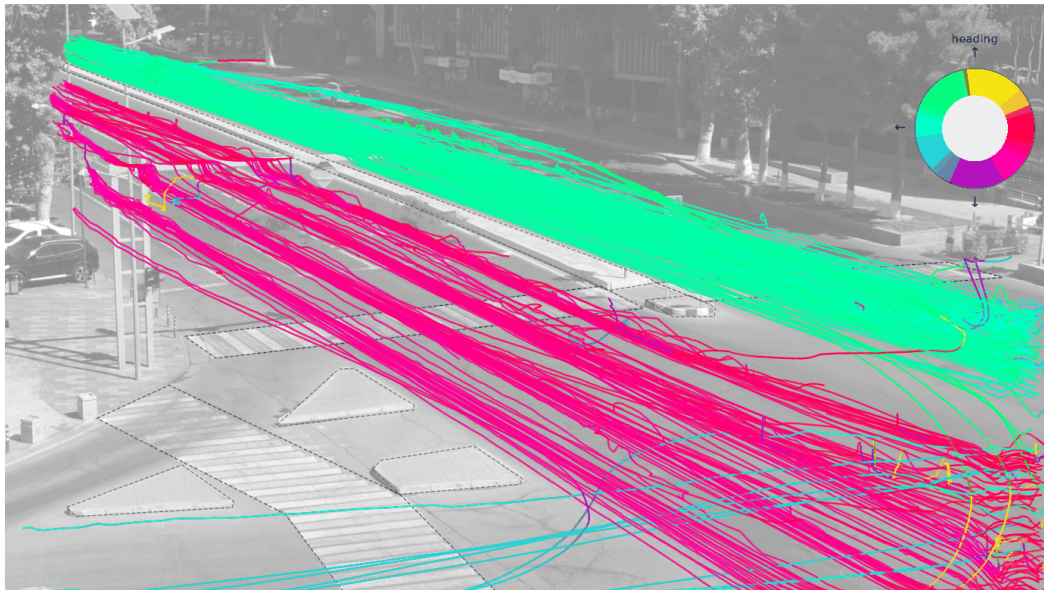
C3905 · pedestrians



Тепловые карты: где едут машины (верх) и где ходят люди (низ).

C3896 · midday — 300 vehicle tracks with the largest net displacement, coloured by heading

Ground point path per track (every 3rd frame), dashed = crosswalks and islands of the scene geometry



Траектории машин, цвет = направление движения.

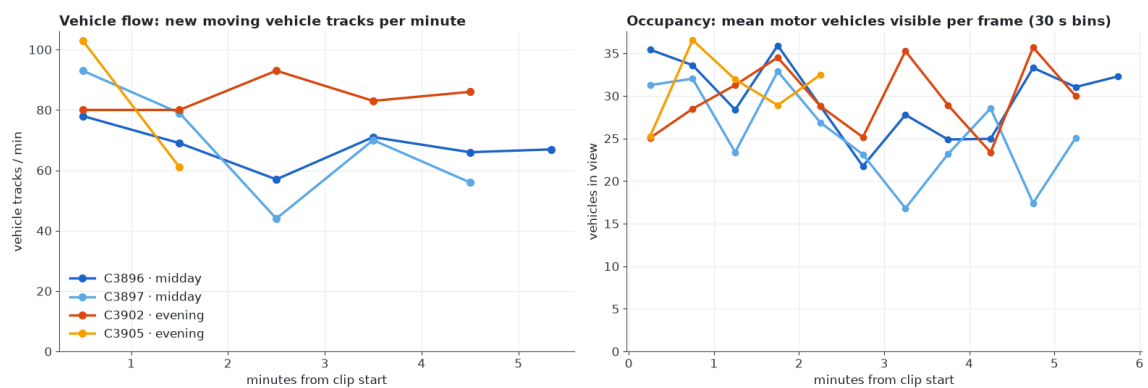
Signal cycle read from pixels: vehicle head on the median and pedestrian head on the left pole

vehicle red vehicle yellow vehicle green not read pedestrian green pedestrian not green



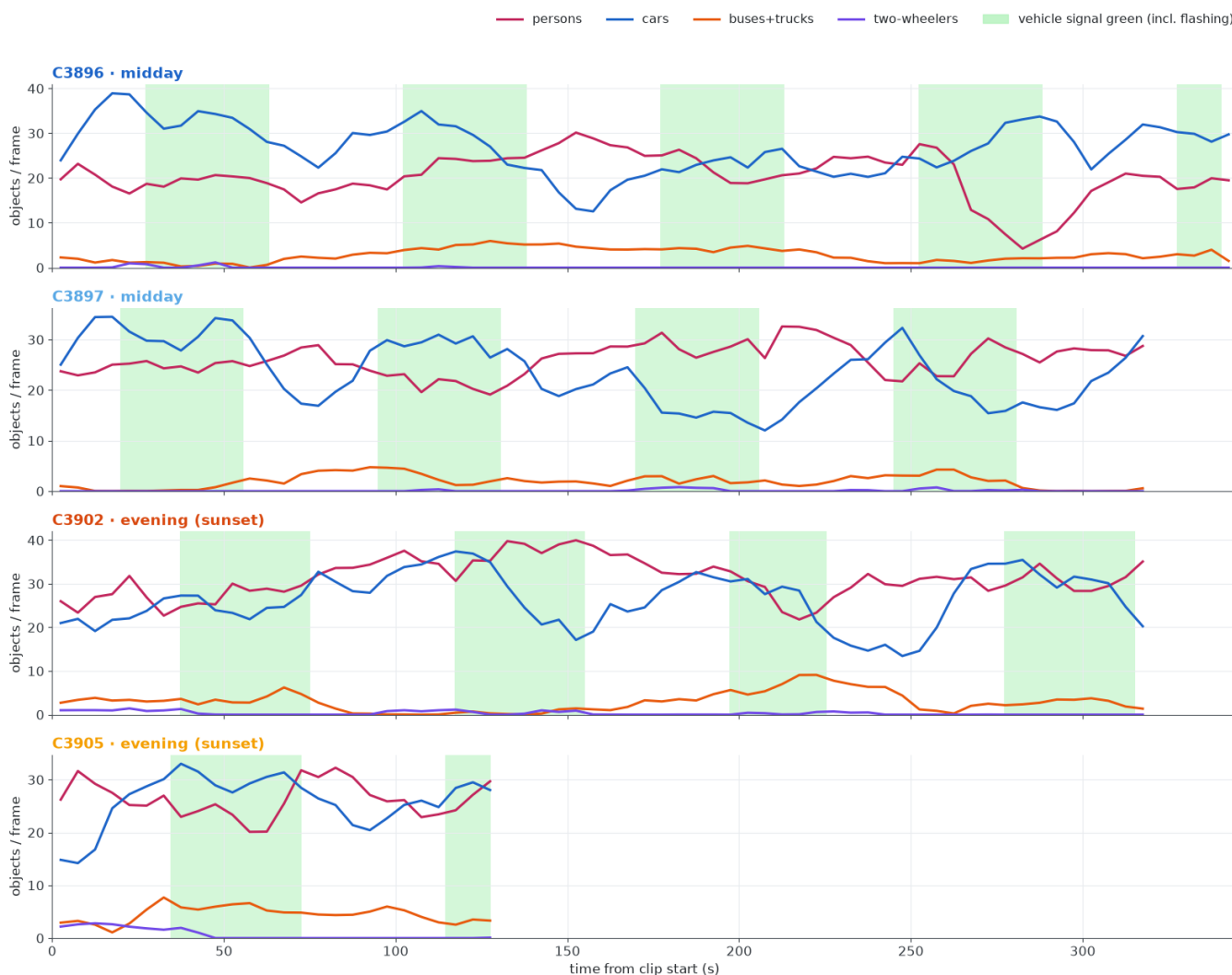
Цикл светофора: 75.0 с днём, 80.0 с вечером; пешеходный зелёный стартует вместе с транспортным.

Traffic density over time (clock time within a clip; clips are not continuous)



Плотность трафика: 68 машин/мин днём, 82 вечером.

Objects per frame over time (5 s bins) with the vehicle-signal green phases



Объекты в кадре по классам во времени (зелёные полосы — зелёный свет).

Выводы, повлиявшие на решение (eda/findings_ru.md):

- 29.97 fps, 4:2:2 10-bit: NVDEC на T4 не декодирует → декод на CPU; берём только опорные кадры (каждый 3-й) — вдвое дешевле.
- 1280 px хватает: события совпадают с 1920 px (0.973), вдвое быстрее.
- Водители в машинах детектируются как «person» (2–6% детекций) → фильтр пассажиров перед правилами для пешеходов.
- Камеру сдвигали на 50–100 px → регистрация каждого видео на опорный вид.
- Светофор читается из пикселей, цикл фиксированный → red_light и stop_line по состоянию лампы.
- Вечером трафик плотнее (82 против 68 машин/мин), детекций не меньше → одна настройка на день и вечер.

Разметка команды (CVAT, 4 видео, 18.4 мин) — реальные частоты классов вместо заглушек:

Класс	Событий	Доля событий	Суммарно, с
jaywalking	31	41.3%	660
solid_line_crossing	10	13.3%	86
congestion	9	12.0%	240
failure_to_yield	9	12.0%	17
stopped_vehicle	5	6.7%	937
stop_line	4	5.3%	123

Класс	Событий	Доля событий	Суммарно, с
illegal_turn	4	5.3%	19
red_light	2	2.7%	4
illegal_u_turn	1	1.3%	3
accident, near_miss, wrong_way, road_obstacle, fire_smoke	0	0%	0

5. Результаты и абляции (реальные числа)

Качество на нашей разметке (официальный evaluate.py, 4 видео)

Класс	F1@0.3	F1@0.5	F1@0.7	Среднее	TP/FP/FN @0.5
jaywalking	0.667	0.444	0.286	0.466	14/18/17
failure_to_yield	0.320	0.320	0.240	0.293	4/12/5
stop_line	0.545	0.545	0.364	0.485	3/4/1
red_light	0.667	0.667	0.000	0.444	1/0/1
stopped_vehicle	0.889	0.889	0.667	0.815	4/0/1
congestion	0.421	0.421	0.316	0.386	4/6/5
solid_line_crossing	0.222	0.111	0.111	0.148	1/7/9

Score A по 9 размеченным классам = 0.338 (0.434 по нашим 7). Это in-sample (пороги подбирались на этих же видео) и по одному разметчику. Два человека на одном видео совпадают на 0.54–0.60 (jaywalking) и 0.80 (failure_to_yield) — это потолок для любой модели.

Абляции

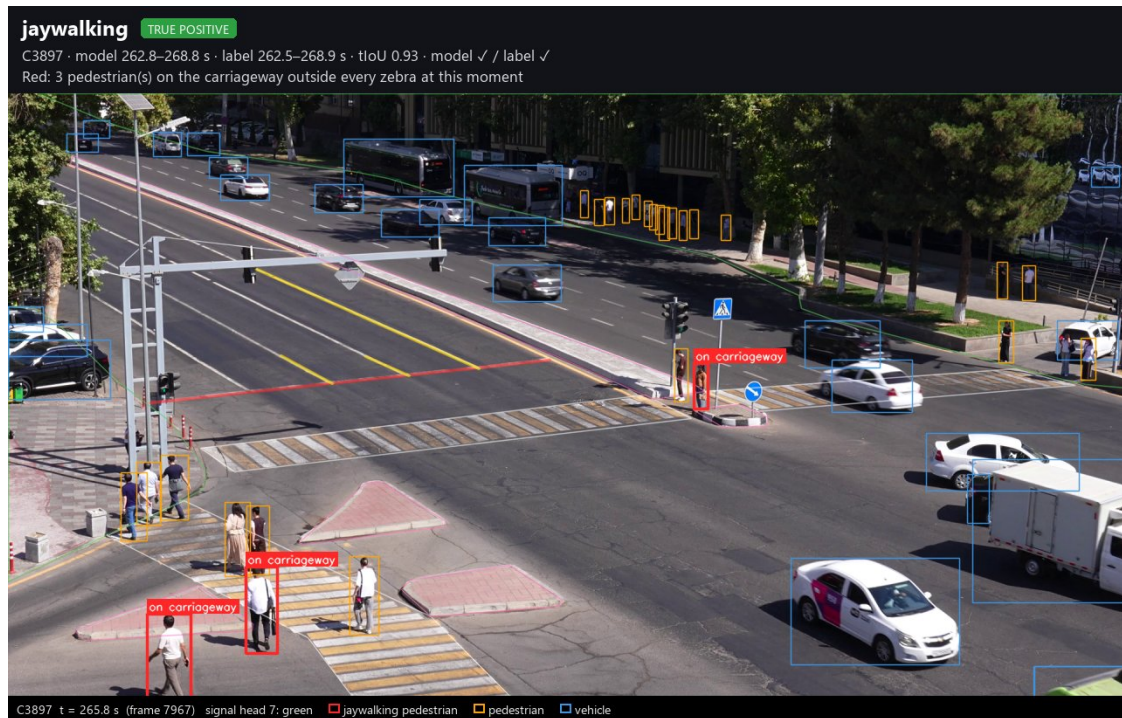
Что сравнивали	Вариант А	Вариант В	Итог
Декодирование 4K (T4, C3905)	cv2.read каждого кадра: 1.41× длительности	PyAV только опорные кадры → 1920 px: 0.70×	В: каждый 3-й кадр вдвое дешевле
Детектор (T4, FP16, каждый 3-й кадр)	YOLO26m@1920: 0.45×	YOLO26m@1280: 0.24×; YOLO26s@960: 0.13×	YOLO26m@1280
Разрешение для событий	1920 px	1280 px	События совпадают (jaywalking 0.973, остальные 0.91–1.00)
Правила (dev Score A, 9 классов)	исходные: 0.308	после настройки: 0.338	jaywalking 0.41→0.47, stop_line 0.33→0.49, FTY 0.26→0.29, solid_line 0.12→0.15 при 8 срабатываниях вместо 57
Part B: TTC в пикселях	изотропный TTC: тревога 73% времени, 3.6/мин	TTC в осях рамки, без окклюзий и попутного следования	финал (640 px): 0.6% времени, 1 тревога за 2 мин (C3905)
Part B: скорость/детерминизм	детекция 1280 + адаптивное прореживание: 2 прогона дали разные кривые	детекция 640 px, только аварийный тормоз	T4: 2.59–2.69× из 3×, два прогона побайтно одинаковы

Прогон на сэмплах (predictions_samples.json)

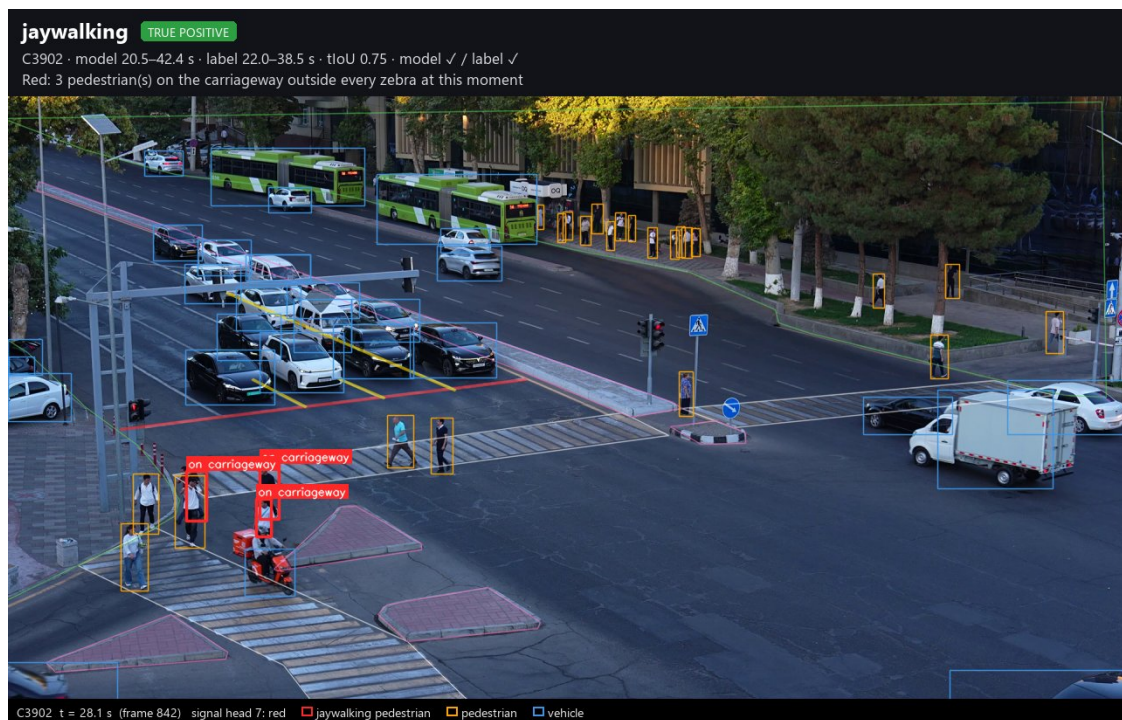
Видео	Длительность, с	Событий	По классам	Тревог Part B	Время на ноутбуке (GTX 1650), ×
C3896	340.34	22	congestion 4, stopped_vehicle 1, jaywalking 9, stop_line 2, red_light 1, solid_line_crossing 4, failure_to_yield 1	27	3.72
C3897	317.82	25	jaywalking 12, stopped_vehicle 1, stop_line 3, solid_line_crossing 2, congestion 2, failure_to_yield 5	39	3.04
C3902	317.82	17	congestion 3, stopped_vehicle 1, jaywalking 7, failure_to_yield 3, solid_line_crossing 2, stop_line 1	28	2.94
C3905	127.63	14	jaywalking 4, stopped_vehicle 1, failure_to_yield 7, congestion 1, stop_line 1	15	2.99

Время для сайта: на Kaggle T4 (как у жюри) видео занимает 2.6–2.7× длительности при лимите 3×; ноутбук медленнее. Кривая риска в predictions_samples.json полная (прогон без ограничения времени); на T4 с 4 CPU Part B останавливает детекцию в начале видео, чтобы не выйти за лимит, — события Part A от этого не меняются.

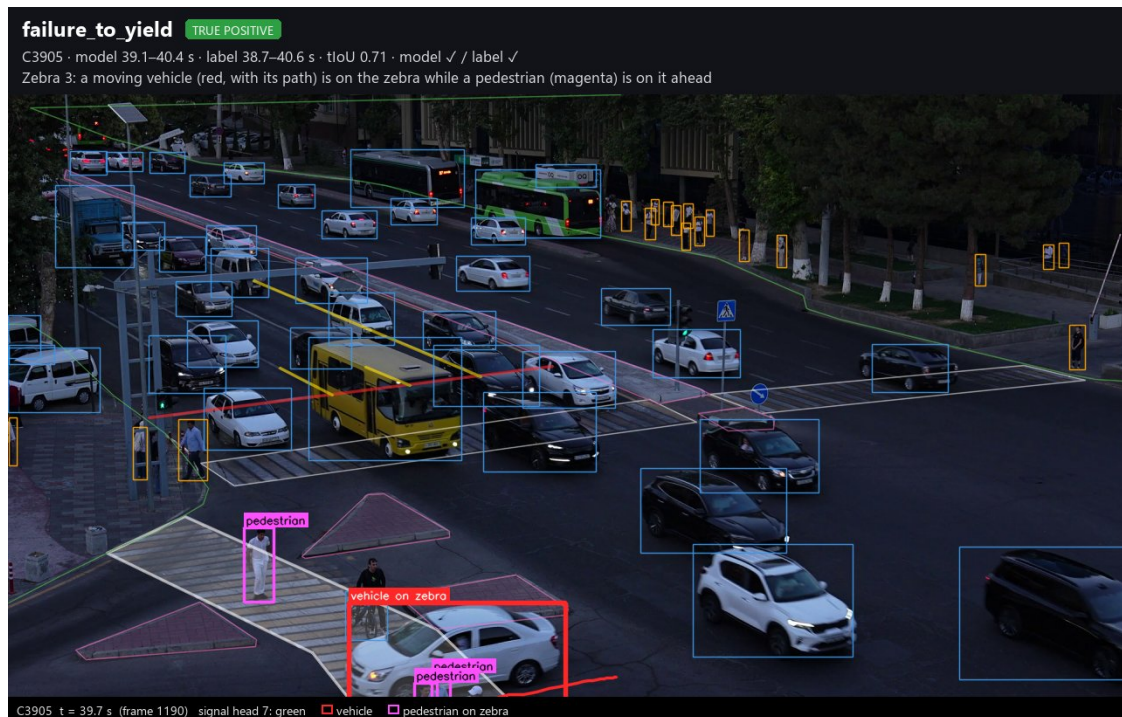
Примеры классов (results/examples/) и честные ошибки (results/failures/)



jaywalking · C3897 — Переход вне зебры, C3897: модель 262.8–268.8 s, разметка 262.5–268.9 s (tIoU 0.93). Красные рамки: пешеходы на проезжей части вне зебр.



jaywalking · C3902 — Переход вне зебры, C3902: модель 20.5–42.4 s, разметка 22.0–38.5 s (tIoU 0.75). Красные рамки: пешеходы на проезжей части вне зебр.



failure_to_yield · C3905 — Непропуск пешехода на зебре 3, C3905: модель 39.1–40.4 s, разметка 38.7–40.6 s. Красным: машина и её путь, пурпурным: пешеход на той же зебре.



failure_to_yield · C3896 — Непропуск пешехода на зебре 3, C3896: модель 179.7–180.3 s, разметка 179.6–180.4 s. Красным: машина и её путь, пурпурным: пешеход на той же зебре.



stop_line · C3905 — Заезд за стоп-линию, C3905: модель 75.6–114.5 s, разметка 76.8–119.3 s. На красный сигнал головы 7 машина в красной рамке стоит за стоп-линией 4.



stop_line · C3896 — Заезд за стоп-линию, C3896: модель 291.5–327.3 s, разметка 299.0–331.0 s. На красный сигнал головы 7 машина в красной рамке стоит за стоп-линией 4.



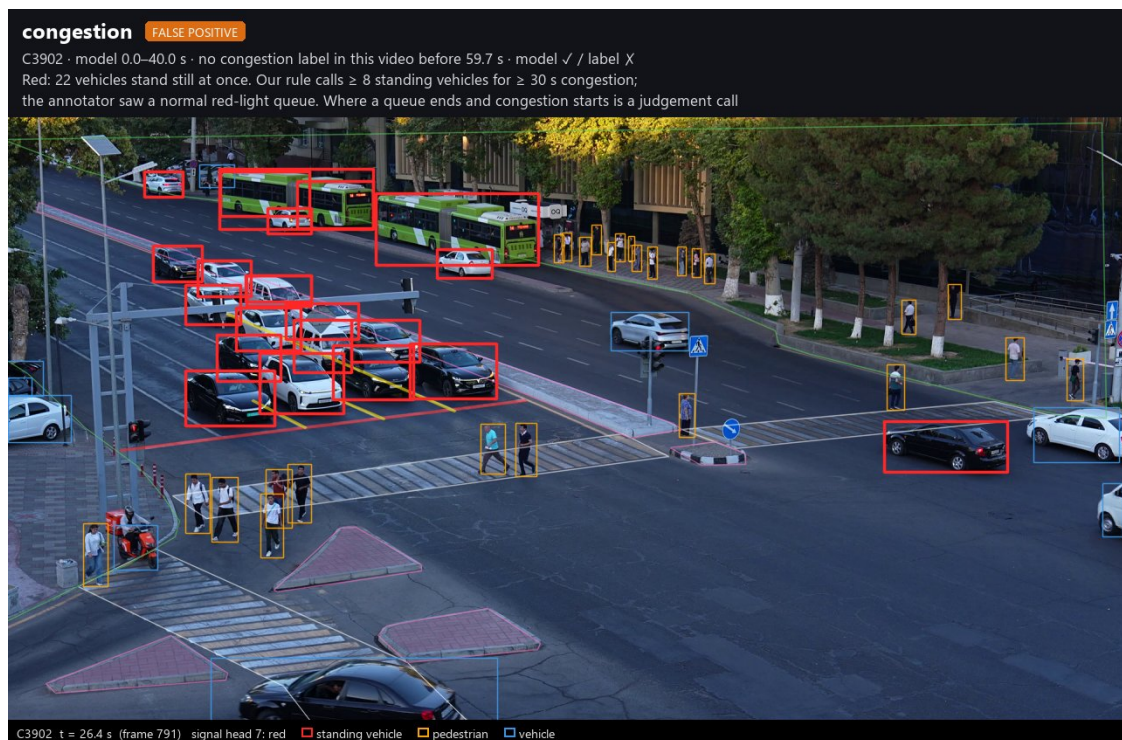
FN · solid_line_crossing · C3896 — Пропуск пересечения сплошной (C3896, разметка 122.7–125.8 с). Машина 2.9 с едет по линии, но по трекам до и после она с одной стороны, и проверка смены стороны её отбрасывает. Эта проверка сократила ложные срабатывания с 57 до 8 на dev-видео; такие пропуски — её цена.



FN · jaywalking · C3897 — Переход вне зебры нарезан иначе (C3897). В разметке одно событие на 122 с (126.5–248.4 с), модель режет его на 5 кусков: между ними пешеход на проезжей части вне зебры не детектируется (этот кадр, t=195 с). Лучший tIoU 0.36, поэтому при пороге 0.5 это 1 пропуск и 5 ложных срабатываний.



FP · failure_to_yield · C3897 — «Ложное срабатывание» не пропуска на зебре 3 (C3897, 91.8–94.7 с). Разметки нет, но на кадре машина едет по зебре 3, пока на ней пешеходы: возможно, реальное неразмеченное событие. Зебра 3 даёт большую часть наших FP в этом классе.



FP · congestion · C3902 — Расхождение по затору (C3902, модель 0–40 с, разметки нет). Одновременно стоят 22 машин, и правило (≥ 8 стоящих машин ≥ 30 с) срабатывает; аннотатор счёл это обычной очередью на светофоре. Затор — наш наименее согласованный с разметкой класс (F1 0.39).



FP · stop_line · C3897 — «Ложный» заезд за стоп-линию (C3897, 58.8–94.8 с). Модель видит машину за стоп-линией 4 на красный головы 7; аннотатор в этом видео заездов не отмечал. На кадре машины явно стоят за линией на красный, так что это похоже на реальное нарушение, пропущенное в разметке, а не на ошибку модели.

6. Отчёт (одна страница для сайта)

Что сработало

- Декодирование только опорных кадров (GOP IBBP) — каждый 3-й кадр почти вдвое дешевле; весь конвейер укладывается в 2.6–2.7× длительности на T4 при лимите 3×.
- Регистрация каждого видео на опорный вид (SIFT + RANSAC) — геометрия сцены рисуется один раз и переносится на любое видео этой камеры.
- Состояние светофора из пикселей — основа red_light и stop_line.
- Своя разметка в CVAT (событие = трек) и официальный evaluate.py на ней — каждое изменение правил проверялось цифрами.
- Шкала времени для jaуwalking: по 0.1-секундной шкале модель совпадает с разметкой так же, как два человека между собой (~0.88).

Что не сработало (честно)

- Автопредразметка для людей: мелкие рамки и решения по стоп-кадрам — команда её отклонила, разметили вручную.
- solid_line_crossing: низкое качество (0.15). Нижняя точка рамки в наклонном ракурсе смещена, подбор порогов (~100 вариантов) не помог; оставили строгое правило «сменил сторону линии».
- illegal_turn / illegal_u_turn не выдаём: camera.md нет, а один и тот же манёвр размечен двумя разными классами.
- Part B: TTC в пикселях срабатывал в 73% времени (плотный трафик, окклюзии) — пришлось фильтровать; проверить заблаговременность на настоящих авариях не на чем — в сэмплах их нет. Отдельный опыт на датасете ACCIDENT дал AP 0.05 на реалистичной смеси.
- Первая версия Part B с адаптивным прореживанием давала разные кривые риска в двух прогонах — заменили на фиксированный шаг с аварийным тормозом.

Что сделали бы дальше

- Второй независимый разметчик на каждое видео и единые правила спорных случаев — сейчас разница ± 0.03 в пределах шума разметки.
- Проекция на плоскость дороги (метрическая гомография) для сплошных линий и скоростей.
- Классы поворотов после ответа организаторов про запрещённые манёвры.
- Обученная модель аварий/near-miss на открытых данных для Part B.

Live demo (30% оценки сайта) — как подключить настоящий инференс

- В папке demo/ — готовое приложение Gradio: загрузка .mp4 (≤ 60 МБ, ≤ 2 мин) → наш конвейер → таблица событий, таймлайн по классам, кривая риска с порогом 0.5, видео с разметкой, JSON для скачивания. Прогресс показывается по этапам.
- Проверено: 20-секундный клип C3905 на CPU — 69 с на 12 потоках ноутбука (3.5× длительности), найдены stopped_vehicle и failure_to_yield, кривая риска и видео с разметкой.
- Деплой: demo/DEPLOY.txt — Hugging Face Space (Gradio, CPU basic): скопировать app.py, requirements.txt, packages.txt, src/ и scene/ из репозитория; веса скачиваются при старте.
- На бесплатном Space 2 CPU: 2-минутный клип ≈ 10 минут. Реалистично: на странице просить клипы 20–40 с (указать это рядом с кнопкой), либо включить платный «CPU upgrade» (8 vCPU) на время судейства.
- Встроить в сайт: iframe на адрес Space или свой React-интерфейс через @gradio/client (endpoint /analyze, 6 выходов: статус, таблица, таймлайн, риск, видео, JSON с events и risk).
- Фиктивный поток загрузки на сайте (setInterval + defaultUploadEvents + синусоида риска) нужно убрать полностью.
- Для другой камеры модель честно работает только по stopped_vehicle и congestion (нет геометрии сцены) — приложение пишет это в статусе.
- Space засыпает без посещений: перед судейством открыть его и прогнать один короткий клип.

7. Файлы кита и форматы

Папка / файл	Что это	Куда на сайте
videos/annotated_C3896.mp4 ... C3905.mp4	Видео с разметкой нашей моделью: геометрия сцены, люди/машины, светофор, таймлайн «модель vs разметка» (1280×872, 10 fps, H.264)	Sample results: плеер для каждого сэмпла
data/events_<video>.json	События модели: start_sec, end_sec, label, desc, desc_ru, type (warning/danger/critical)	Таймлайн под плеером, клик → jumpTo(start_sec)
data/risk_<video>.json	Кривая риска Part B: [t_sec, risk] 2 раза в секунду, порог тревоги 0.5	График R(t) под плеером
data/samples_summary.json	Сводка по сэмплам: события по классам, тревоги, время	Карточки результатов
eda/*.png, eda/*.json	Графики EDA и данные к ним	EDA-раздел; JSON — для интерактивных графиков
results/examples, failures, *.json	Кадры примеров по классам и ошибок + подписи RU/EN + метрики по классам	Examples of each class, failure cases
demo/	Настоящее live-demo (Gradio): загрузка ≤ 2 мин → события, таймлайн, кривая риска, видео с разметкой	Live demo (iframe или @gradio/client)
build_site_data.py	Пересобирает data/ из predictions_samples.json	—

Пример события (data/events C3905.json):

```
{"start_sec": 75.64, "end_sec": 114.48, "label": "stop_line",  
"desc": "Vehicle stopped past stop line 4 while its signal is red",  
"desc_ru": "Машина остановилась за стоп-линией 4 на красный", "type": "danger"}
```

Полей track и conf у нас нет: события — это сегменты класса по времени (так размечают организаторы), а не отдельные объекты. Не придумывайте их — уберите колонку или покажите класс и длительность.

8. Ссылки для раздела Links

- Репозиторий (код, README, predictions_samples.json): <https://github.com/AsanAshirov/WestCV> — тер v1.0.0. Репозиторий должен быть публичным к сдаче.
- Beca: скачиваются weights/download.sh с <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo26m.pt> (44 МБ, sha256 проверяется).
- predictions_samples.json: https://github.com/AsanAshirov/WestCV/blob/v1.0.0/predictions_samples.json
- Live demo: адрес Hugging Face Space после деплоя (demo/DEPLOY.txt).